

Самостійна робота №4: Доступність (Accessibility) у веб-застосунках

Тема: Доступність (Accessibility) у веб-застосунках

Мета: Зрозуміти принципи веб-доступності, вивчити стандарт WCAG, навчитися використовувати семантичний HTML, ARIA-атрибути та забезпечувати клавіатурну навігацію.

Кількість годин: 3

Теоретичний матеріал

Що таке веб-доступність (a11y)

Веб-доступність (accessibility, скорочено a11y) - це практика створення веб-сайтів, доступних для всіх людей, включаючи людей з обмеженими можливостями: порушення зору, слуху, моторики, когнітивні обмеження. Доступність також покращує UX для всіх користувачів: мобільних, з повільним інтернетом, похилого віку.

WCAG (Web Content Accessibility Guidelines)

WCAG - міжнародний стандарт доступності веб-контенту. Базується на чотирьох принципах (POUR): **Perceivable** (сприйнятність) - контент можна сприйняти різними способами; **Operable** (керованість) - інтерфейсом можна керувати різними засобами; **Understandable** (зрозумілість) - контент та інтерфейс зрозумілі; **Robust** (надійність) - контент працює з різними допоміжними технологіями. Рівні відповідності: А (мінімальний), АА (рекомендований), ААА (максимальний).

Семантичний HTML

Семантичні елементи передають значення контенту: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`. Скрінрідери використовують семантику для навігації. Замість `<div onclick>` слід використовувати `<button>`, замість `<div>` для навігації - `<nav>`. Правильна ієрархія заголовків (`<h1>` – `<h6>`) створює структуру документа.

ARIA-атрибути

ARIA (Accessible Rich Internet Applications) - набір атрибутів для покращення доступності динамічного контенту. Основні атрибути: `role` (роль елемента), `aria-label` (текстова мітка), `aria-labelledby` (посилання на елемент з міткою), `aria-describedby` (додатковий опис), `aria-hidden` (приховати від скрінрідерів), `aria-live` (оголошення змін), `aria-expanded` (стан розгортання). Правило: не використовуйте ARIA, якщо можна використати нативний HTML-елемент.

Клавіатурна навігація

Всі інтерактивні елементи мають бути доступні з клавіатури. Tab - переміщення між елементами, Enter/Space - активація, Escape - закриття. `tabindex="0"` - додає елемент у порядок табуляції, `tabindex="-1"` - дозволяє програмний фокус. Стили фокусу (`:focus-visible`) мають бути видимими. Focus trap - утримання фокусу всередині модальних вікон.

Контрольні питання

1. Що означає аббревіатура WCAG і які чотири принципи лежать в основі стандарту?
2. Яка різниця між рівнями відповідності A, AA та AAA?
3. Чому семантичний HTML важливий для доступності? Наведіть приклади семантичних елементів.
4. В яких випадках слід використовувати ARIA-атрибути, а коли краще обійтися нативним HTML?
5. Як працює атрибут `aria-live` і для чого він використовується?
6. Що таке focus trap і коли його потрібно реалізовувати?
7. Як перевірити доступність веб-застосунку? Які інструменти для цього існують?

Практичне завдання

Проаналізуйте будь-яку сторінку вашого React-застосунку (або створіть нову) та виконайте:

1. Запустіть аудит доступності за допомогою Lighthouse або axe DevTools.
2. Замініть всі не-семантичні елементи на семантичні (`<div>` на `<nav>`, `<main>`, `<button>` тощо).
3. Додайте `aria-label` до елементів, де це необхідно (іконки-кнопки, поля форм).
4. Переконайтесь, що всі інтерактивні елементи доступні з клавіатури (Tab, Enter).
5. Перевірте контрастність тексту (мінімум 4.5:1 для WCAG AA).

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.
4. Виконати практичне завдання.

Рекомендовані джерела

- [WCAG 2.2 - W3C](#) - офіційний стандарт WCAG
- [MDN - Accessibility](#) - гайд по доступності від MDN
- [WAI-ARIA Authoring Practices Guide](#) - патерни реалізації ARIA
- [axe DevTools](#) - інструмент для аудиту доступності у браузері
- [WebAIM - Contrast Checker](#) - перевірка контрастності кольорів
- [A11y Project](#) - спільнота та ресурси з доступності